

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12
SOLID PRINCIPLE



Disusun Oleh:

M Rizki Ruswan 105224018

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Pada tugas ini saya melakukan analisis dan perancangan ulang terhadap sistem akademik kampus (SIKAD) yang mengalami berbagai permasalahan arsitektur perangkat lunak. Sistem lama memiliki sebuah kelas besar bernama SistemKRSManager yang bertanggung jawab atas banyak fungsi sekaligus, seperti validasi KRS, perhitungan UKT, pencetakan PDF, serta penyimpanan data ke database. Akibatnya, perubahan kecil pada satu fitur sering kali menyebabkan kerusakan pada fitur lainnya.

Selain itu, sistem lama juga mengalami berbagai pelanggaran terhadap prinsip-prinsip SOLID. Perhitungan UKT dilakukan menggunakan banyak percabangan if-else, antarmuka mata kuliah memaksa seluruh kelas mengimplementasikan method yang tidak diperlukan, dan koneksi database terikat langsung pada MySQL sehingga sulit untuk dimigrasikan ke teknologi lain.

Untuk mengatasi masalah tersebut, saya melakukan refactoring sistem dengan menerapkan lima prinsip SOLID yaitu Single Responsibility Principle (SRP), Open Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP). Hasil perancangan ulang menghasilkan sistem yang lebih modular, mudah dipelihara, mudah dikembangkan, serta siap menghadapi migrasi database ke Cloud NoSQL dan penambahan skema UKT baru seperti MBKM.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	nama	String	Menyimpan nama mata kuliah.
2	sks	int	Menyimpan jumlah SKS mata kuliah.
3	database	DatabaseConnection	Menyimpan objek koneksi database yang digunakan sistem.
4	validator	krsValidator	Digunakan untuk melakukan validasi data KRS mahasiswa.
5	repository	krsRepository	Digunakan untuk menyimpan data KRS ke database.
6	printer	krsPdfPrinter	Digunakan untuk mencetak atau menghasilkan dokumen PDF KRS.
7	strategi	StrategiKalkulasiUKT	Menyimpan strategi perhitungan UKT yang digunakan.
8	db	DatabaseConnection	Menentukan jenis database yang digunakan (MySQL atau Cloud NoSQL).

9	mk1	MataKuliah	Menyimpan objek mata kuliah teori.
10	mk2	MataKuliah	Menyimpan objek mata kuliah praktikum.
11	mk3	MataKuliah	Menyimpan objek mata kuliah KKN.
12	totalSKS	int	Menyimpan jumlah total SKS yang diambil mahasiswa.
13	ukt	double	Menyimpan hasil perhitungan UKT mahasiswa.
14	data	String	Menyimpan data KRS yang akan disimpan ke database.

III. Constructor dan Method

No	Nama Method	Jenis Method	Fungsi
1	MataKuliah(String nama, int sks)	Constructor	Membuat objek mata kuliah dengan nama dan jumlah SKS tertentu.
2	getNama()	Getter	Mengambil nilai nama mata kuliah.
3	getSks()	Getter	Mengambil nilai jumlah SKS mata kuliah.
4	tampilkanInfo()	Abstract Method	Menampilkan informasi mata kuliah sesuai jenisnya.
5	MataKuliahTeori(String nama, int sks)	Constructor	Membuat objek mata kuliah teori.
6	tampilkanInfo()	Override Method	Menampilkan informasi mata kuliah teori.
7	MataKuliahPraktikum(String nama, int sks)	Constructor	Membuat objek mata kuliah praktikum.
8	tampilkanInfo()	Override Method	Menampilkan informasi mata kuliah praktikum.
9	alokasiAsistenLab()	Interface Method	Mengalokasikan asisten laboratorium untuk praktikum.

10	cekPeralatanPraktikum()	Interface Method	Memeriksa peralatan yang digunakan dalam praktikum.
11	MataKuliahKKN(String nama, int sks)	Constructor	Membuat objek mata kuliah KKN.
12	tampilkanInfo()	Override Method	Menampilkan informasi mata kuliah KKN.
13	hitungUKT()	Interface Method	Menghitung besaran UKT sesuai jalur mahasiswa.
14	hitungUKT() pada uktReguler	Override Method	Menghitung UKT mahasiswa jalur reguler.
15	hitungUKT() pada uktKaryawan	Override Method	Menghitung UKT mahasiswa jalur karyawan.
16	hitungUKT() pada uktInternasional	Override Method	Menghitung UKT mahasiswa jalur internasional.
17	hitungUKT() pada uktBidikmisi	Override Method	Menghitung UKT mahasiswa jalur bidikmisi.
18	hitungUKT() pada uktMbkm	Override Method	Menghitung UKT mahasiswa program MBKM.
19	saveKRS(String data)	Interface Method	Menyimpan data KRS ke media penyimpanan.
20	saveKRS(String data) pada MySQLDatabaseConnection	Override Method	Menyimpan data KRS ke database MySQL.
21	saveKRS(String data) pada CloudNoSQLConnection	Override Method	Menyimpan data KRS ke database Cloud NoSQL.
22	validasiSKS(int totalSKS)	Procedural	Memvalidasi jumlah SKS yang diambil mahasiswa.
23	cetakPDF()	Procedural	Membuat dan mencetak dokumen PDF KRS.

24	krsRepository(DatabaseConnection database)	Constructor	Menginisialisasi objek repository dengan koneksi database tertentu.
25	simpan(String data)	Procedural	Menyimpan data KRS melalui repository.
26	krsService(krsValidator validator, krsRepository repository, krsPdfPrinter printer, StrategiKalkulasiUKT strategi)	Constructor	Menginisialisasi seluruh komponen layanan KRS.
27	prosesKRS(int totalSKS)	Procedural	Menjalankan proses validasi, perhitungan UKT, penyimpanan data, dan pencetakan PDF.
28	main(String[] args)	Static Method	Menjalankan program utama dan menguji seluruh fitur sistem.

IV. Dokumentasi dan Pembahasan Code

1.

```

1  public class CloudNoSQLConnection implements DatabaseConnection {
2
3      @Override
4      public void saveKRS(String data) {
5          System.out.println(x: "Menyimpan KRS ke Cloud NoSQL...");
6          System.out.println(data);
7      }
8  }

```

Penjelasan:

Pada bagian ini saya membuat class `CloudNoSQLConnection` yang mengimplementasikan interface `DatabaseConnection` sebagai bentuk penerapan prinsip Dependency Inversion Principle (DIP). Class ini berfungsi sebagai representasi koneksi ke database Cloud NoSQL yang digunakan untuk menyimpan data KRS mahasiswa. Di dalam class terdapat method `saveKRS(String data)` yang merupakan implementasi dari method yang didefinisikan pada interface `DatabaseConnection`. Method ini menerima parameter berupa data KRS yang akan disimpan, kemudian menampilkan pesan *"Menyimpan KRS ke Cloud NoSQL..."* sebagai simulasi proses penyimpanan data ke database cloud. Selanjutnya, data yang diterima juga ditampilkan menggunakan `System.out.println(data)`. Dengan adanya class ini, sistem dapat menggunakan Cloud

NoSQL sebagai media penyimpanan tanpa perlu mengubah kode pada bagian layanan atau logika bisnis, sehingga proses migrasi database menjadi lebih mudah, fleksibel, dan tidak memengaruhi komponen lain dalam aplikasi.

2.

```
THT > src > DatabaseConnection.java > DatabaseConnection
1 public interface DatabaseConnection {
2     void saveKRS(String data);
3 }
```

Penjelasan:

Pada bagian ini saya membuat interface `DatabaseConnection` yang berfungsi sebagai abstraksi untuk proses penyimpanan data KRS ke database. Interface ini mendeklarasikan satu method yaitu `saveKRS(String data)` yang menerima parameter berupa data KRS dalam bentuk `String`. Karena merupakan interface, method tersebut tidak memiliki implementasi dan hanya mendefinisikan kontrak yang harus dipenuhi oleh setiap class yang mengimplementasikannya. Dengan adanya interface ini, sistem tidak bergantung langsung pada jenis database tertentu, melainkan hanya bergantung pada abstraksi. Oleh karena itu, berbagai jenis database seperti `MySQLDatabaseConnection` maupun `CloudNoSQLConnection` dapat mengimplementasikan method `saveKRS()` sesuai kebutuhan masing-masing. Pendekatan ini membuat sistem lebih fleksibel, mudah dikembangkan, serta mendukung penerapan prinsip `Dependency Inversion Principle (DIP)` karena modul tingkat tinggi tidak bergantung pada implementasi konkret database.

3.

```
1 public class MySQLDatabaseConnection implements DatabaseConnection {
2
3     @Override
4     public void saveKRS(String data) {
5         System.out.println(x: "Menyimpan KRS ke MySQL...");
6         System.out.println(data);
7     }
8 }
```

Penjelasan:

Pada bagian ini saya membuat class `MySQLDatabaseConnection` yang mengimplementasikan interface `DatabaseConnection`. Class ini berfungsi untuk menangani proses penyimpanan data KRS ke database `MySQL`. Karena mengimplementasikan interface `DatabaseConnection`, class ini wajib menyediakan implementasi untuk method `saveKRS(String data)` yang telah didefinisikan sebelumnya. Di dalam method tersebut, sistem menampilkan pesan "Menyimpan KRS ke MySQL..." sebagai simulasi proses penyimpanan data ke database `MySQL`. Setelah

itu, data KRS yang diterima melalui parameter data ditampilkan menggunakan `System.out.println(data)`. Dengan adanya class ini, sistem memiliki implementasi konkret untuk penyimpanan data menggunakan MySQL tanpa harus mengubah logika bisnis yang terdapat pada class lain. Selain itu, penggunaan interface memungkinkan sistem untuk beralih ke jenis database lain, seperti Cloud NoSQL, dengan mudah hanya dengan mengganti implementasi yang digunakan. Pendekatan ini mendukung prinsip Dependency Inversion Principle (DIP) sehingga aplikasi menjadi lebih fleksibel, modular, dan mudah dipelihara.

4.

```
THT > src > MataKuliah.java > MataKuliah
1  public abstract class MataKuliah {
2
3      protected String nama;
4      protected int sks;
5
6      public MataKuliah(String nama, int sks) {
7          this.nama = nama;
8          this.sks = sks;
9      }
10
11     public String getNama() {
12         return nama;
13     }
14
15     public int getSks() {
16         return sks;
17     }
18
19     public abstract void tampilkanInfo();
20 }
```

Penjelasan:

Pada bagian ini saya membuat class abstrak `MataKuliah` yang berfungsi sebagai class induk (superclass) untuk seluruh jenis mata kuliah yang ada dalam sistem, seperti mata kuliah Teori, Praktikum, dan KKN. Class ini memiliki dua atribut yaitu `nama` bertipe `String` yang digunakan untuk menyimpan nama mata kuliah, serta `sks` bertipe `int` yang digunakan untuk menyimpan jumlah SKS mata kuliah tersebut. Selanjutnya, saya membuat constructor `MataKuliah(String nama, int sks)` yang berfungsi untuk menginisialisasi nilai kedua atribut saat objek turunan dibuat. Selain itu, terdapat method `getNama()` dan `getSks()` yang berfungsi sebagai getter untuk mengambil nilai atribut `nama` dan `SKS`. Pada class ini juga terdapat method abstrak `tampilkanInfo()` yang tidak memiliki implementasi dan harus diimplementasikan oleh setiap class turunan sesuai kebutuhan masing-masing. Dengan menggunakan class abstrak, saya dapat

menyimpan atribut dan perilaku umum yang dimiliki seluruh mata kuliah dalam satu tempat sehingga kode menjadi lebih terstruktur, mengurangi duplikasi, serta mendukung penerapan konsep Inheritance dan Polymorphism dalam pemrograman berorientasi objek.

5.

```
THT > src > MataKuliahKKN.java > MataKuliahKKN
1  public class MataKuliahKKN extends MataKuliah {
2
3      public MataKuliahKKN(String nama, int sks) {
4          super(nama, sks);
5      }
6
7      @Override
8      public void tampilkanInfo() {
9          System.out.println("Mata Kuliah KKN : " + nama);
10     }
11 }
```

Penjelasan:

Pada bagian ini saya membuat class `MataKuliahKKN` yang merupakan turunan dari class abstrak `MataKuliah`. Class ini digunakan untuk merepresentasikan mata kuliah jenis Kuliah Kerja Nyata (KKN) dalam sistem akademik. Pada constructor `MataKuliahKKN(String nama, int sks)`, saya menggunakan keyword `super(nama, sks)` untuk memanggil constructor milik class induk sehingga atribut `nama` dan `sks` dapat diinisialisasi secara langsung. Selanjutnya, class ini melakukan override terhadap method abstrak `tampilkanInfo()` yang terdapat pada class `MataKuliah`. Implementasi method tersebut digunakan untuk menampilkan informasi mata kuliah KKN dengan format *"Mata Kuliah KKN : "* diikuti nama mata kuliah yang tersimpan pada atribut `nama`. Dengan adanya class ini, sistem dapat membedakan jenis mata kuliah KKN dari jenis mata kuliah lainnya tanpa perlu menambahkan method yang tidak relevan seperti pengelolaan laboratorium. Penerapan ini mendukung konsep Inheritance, Polymorphism, serta memenuhi prinsip Liskov Substitution Principle (LSP) karena objek `MataKuliahKKN` dapat digunakan sebagai pengganti objek `MataKuliah` tanpa mengubah perilaku sistem.

6.

```

THT > src > MataKuliahPraktikum.java > MataKuliahPraktikum > MataKuliahPraktikum(String, int)
1  public class MataKuliahPraktikum extends MataKuliah
2      implements OperasiPraktikum {
3
4      public MataKuliahPraktikum(String nama, int sks) {
5          super(nama, sks);
6      }
7
8      @Override
9      public void tampilkanInfo() {
10         System.out.println("Mata Kuliah Praktikum : " + nama);
11     }
12
13     @Override
14     public void alokasiAsistenLab() {
15         System.out.println(x: "Asisten Lab Dialokasikan");
16     }
17
18     @Override
19     public void cekPeralatanPraktikum() {
20         System.out.println(x: "Peralatan Praktikum Dicek");
21     }
22 }

```

Penjelasan:

Pada bagian ini saya membuat class MataKuliahPraktikum yang merupakan turunan dari class abstrak MataKuliah dan sekaligus mengimplementasikan interface OperasiPraktikum. Class ini digunakan untuk merepresentasikan mata kuliah yang memiliki kegiatan praktikum sehingga memerlukan pengelolaan laboratorium dan peralatan pendukung. Pada constructor MataKuliahPraktikum(String nama, int sks), saya menggunakan super(nama, sks) untuk menginisialisasi atribut nama dan sks yang dimiliki oleh class induk. Selanjutnya, saya melakukan override terhadap method tampilkanInfo() untuk menampilkan informasi mata kuliah praktikum dengan format "Mata Kuliah Praktikum :" diikuti nama mata kuliah. Karena class ini mengimplementasikan interface OperasiPraktikum, maka saya juga mengimplementasikan method alokasiAsistenLab() yang berfungsi untuk mengalokasikan asisten laboratorium serta method cekPeralatanPraktikum() yang digunakan untuk memeriksa ketersediaan peralatan praktikum. Dengan memisahkan operasi praktikum ke dalam interface tersendiri, hanya mata kuliah yang benar-benar membutuhkan fitur laboratorium yang mengimplementasikan method tersebut. Pendekatan ini mendukung konsep Inheritance, Polymorphism, serta menerapkan Interface Segregation Principle (ISP) karena class tidak dipaksa mengimplementasikan method yang tidak relevan dengan kebutuhannya.

7.

```

THT > src > MataKuliahTeori.java > MataKuliahTeori
1  public class MataKuliahTeori extends MataKuliah {
2
3      public MataKuliahTeori(String nama, int sks) {
4          super(nama, sks);
5      }
6
7      @Override
8      public void tampilkanInfo() {
9          System.out.println("Mata Kuliah Teori : " + nama);
10     }
11 }

```

Penjelasan:

Pada bagian ini saya membuat class MataKuliahTeori yang merupakan turunan dari class abstrak MataKuliah. Class ini digunakan untuk merepresentasikan mata kuliah yang bersifat teori dan tidak memerlukan fasilitas laboratorium maupun kegiatan praktikum. Pada constructor MataKuliahTeori(String nama, int sks), saya menggunakan keyword super(nama, sks) untuk memanggil constructor milik class induk sehingga atribut nama dan sks dapat diinisialisasi secara langsung. Selanjutnya, saya melakukan override terhadap method abstrak tampilkanInfo() yang terdapat pada class MataKuliah. Method ini digunakan untuk menampilkan informasi mata kuliah teori dengan format "Mata Kuliah Teori :" diikuti nama mata kuliah yang tersimpan pada atribut nama. Dengan adanya class ini, sistem dapat membedakan mata kuliah teori dari jenis mata kuliah lainnya seperti Praktikum dan KKN tanpa perlu menambahkan method yang tidak diperlukan. Penerapan class ini mendukung konsep Inheritance dan Polymorphism, serta memenuhi prinsip Liskov Substitution Principle (LSP) karena objek MataKuliahTeori dapat digunakan sebagai pengganti objek MataKuliah tanpa mengubah perilaku dasar sistem.

8.

```

THT > src > OperasiPraktikum.java > OperasiPraktikum
1  public interface OperasiPraktikum {
2
3      void alokasiAsistenLab();
4
5      void cekPeralatanPraktikum();
6  }

```

Penjelasan:

Pada bagian ini saya membuat interface OperasiPraktikum yang digunakan untuk mendefinisikan operasi-operasi khusus yang hanya diperlukan oleh mata kuliah praktikum. Interface ini memiliki dua method, yaitu alokasiAsistenLab() yang

berfungsi untuk mengatur atau mengalokasikan asisten laboratorium, serta cekPeralatanPraktikum() yang digunakan untuk memeriksa ketersediaan dan kondisi peralatan yang dibutuhkan dalam kegiatan praktikum. Karena kedua method tersebut berada di dalam interface, maka method tersebut tidak memiliki implementasi dan harus diimplementasikan oleh class yang membutuhkannya, seperti MataKuliahPraktikum. Dengan memisahkan operasi praktikum ke dalam interface tersendiri, class lain seperti MataKuliahTeori dan MataKuliahKKN tidak perlu mengimplementasikan method yang tidak relevan dengan fungsinya. Pendekatan ini membuat desain sistem menjadi lebih bersih, mengurangi kode yang tidak diperlukan, serta menerapkan prinsip Interface Segregation Principle (ISP) yang menyatakan bahwa sebuah class tidak boleh dipaksa bergantung pada method yang tidak digunakannya.

9.

```
THT > src > StrategiKalkulasiUKT.java > StrategiKalkulasiUKT
1  public interface StrategiKalkulasiUKT {
2
3      double hitungUKT();
4  }
```

Pada bagian ini saya membuat interface StrategiKalkulasiUKT yang digunakan sebagai abstraksi untuk berbagai metode perhitungan Uang Kuliah Tunggal (UKT) berdasarkan jalur masuk mahasiswa. Interface ini memiliki satu method yaitu hitungUKT() yang mengembalikan nilai bertipe double sebagai hasil perhitungan UKT. Karena merupakan interface, method tersebut tidak memiliki implementasi dan hanya mendefinisikan kontrak yang harus dipenuhi oleh setiap class yang mengimplementasikannya. Selanjutnya, interface ini diimplementasikan oleh beberapa class seperti uktReguler, uktKaryawan, uktInternasional, uktBidikmisi, dan uktMbkkm, yang masing-masing memiliki aturan perhitungan UKT yang berbeda. Dengan menggunakan interface ini, sistem dapat memilih strategi perhitungan UKT yang sesuai tanpa perlu menggunakan banyak percabangan if-else. Pendekatan ini membuat kode menjadi lebih fleksibel, mudah dikembangkan, serta mendukung penerapan Open Closed Principle (OCP) karena penambahan jalur mahasiswa baru dapat dilakukan dengan membuat class baru tanpa mengubah kode yang sudah ada.

10.

```

THT > src > uktBidikmisi.java > uktBidikmisi
1  public class uktBidikmisi implements StrategiKalkulasiUKT {
2
3      @Override
4      public double hitungUKT() {
5          return 0;
6      }
7  }

```

Penjelasan:

Pada bagian ini saya membuat class `uktBidikmisi` yang mengimplementasikan interface `StrategiKalkulasiUKT`. Class ini digunakan untuk merepresentasikan skema perhitungan UKT bagi mahasiswa penerima program Bidikmisi. Karena mengimplementasikan interface `StrategiKalkulasiUKT`, class ini wajib menyediakan implementasi untuk method `hitungUKT()`. Pada implementasinya, method tersebut mengembalikan nilai 0, yang menunjukkan bahwa mahasiswa penerima Bidikmisi tidak dikenakan biaya UKT. Dengan membuat skema perhitungan UKT dalam class tersendiri, aturan perhitungan untuk setiap jalur mahasiswa dapat dipisahkan dengan jelas dan tidak bercampur dengan logika jalur lainnya. Pendekatan ini memudahkan pengelolaan serta pengembangan sistem apabila terdapat perubahan kebijakan atau penambahan jalur mahasiswa baru di masa mendatang. Selain itu, penggunaan class ini juga mendukung penerapan Strategy Pattern dan Open Closed Principle (OCP) karena sistem dapat menambahkan strategi perhitungan baru tanpa perlu mengubah kode yang sudah ada.

11.

```

THT > src > uktInternasional.java > uktInternasional
1  public class uktInternasional implements StrategiKalkulasiUKT {
2
3      @Override
4      public double hitungUKT() {
5          return 10000000;
6      }
7  }

```

Penjelasan:

Pada bagian ini saya membuat class `uktInternasional` yang mengimplementasikan interface `StrategiKalkulasiUKT`. Class ini digunakan untuk merepresentasikan skema perhitungan Uang Kuliah Tunggal (UKT) bagi mahasiswa jalur Internasional. Karena mengimplementasikan interface `StrategiKalkulasiUKT`, class ini wajib menyediakan implementasi untuk method `hitungUKT()`. Pada method tersebut, saya mengembalikan nilai 10000000 yang menunjukkan bahwa mahasiswa jalur internasional dikenakan UKT sebesar Rp10.000.000. Nilai tersebut merupakan

simulasi aturan perhitungan UKT yang berbeda dari jalur mahasiswa lainnya. Dengan memisahkan perhitungan UKT ke dalam class tersendiri, setiap jalur masuk memiliki logika perhitungan yang independen sehingga perubahan pada satu jalur tidak akan memengaruhi jalur lainnya. Pendekatan ini membuat kode lebih terstruktur, mudah dipelihara, dan mendukung penerapan Strategy Pattern serta Open Closed Principle (OCP) karena sistem dapat menambahkan skema UKT baru tanpa perlu memodifikasi kode yang sudah ada.

12.

```
THT > src > uktKaryawan.java > uktKaryawan
1  public class uktKaryawan implements StrategiKalkulasiUKT {
2
3      @Override
4      public double hitungUKT() {
5          return 5000000;
6      }
7  }
```

Penjelasan:

Pada bagian ini saya membuat class `uktKaryawan` yang mengimplementasikan interface `StrategiKalkulasiUKT`. Class ini digunakan untuk merepresentasikan skema perhitungan Uang Kuliah Tunggal (UKT) bagi mahasiswa jalur Karyawan. Karena mengimplementasikan interface `StrategiKalkulasiUKT`, class ini wajib menyediakan implementasi untuk method `hitungUKT()`. Pada implementasinya, method tersebut mengembalikan nilai 5000000 yang menunjukkan bahwa mahasiswa jalur karyawan dikenakan UKT sebesar Rp5.000.000. Nilai ini merupakan simulasi biaya kuliah yang ditetapkan untuk jalur karyawan dan berbeda dengan jalur mahasiswa lainnya. Dengan memisahkan logika perhitungan UKT ke dalam class tersendiri, setiap jalur masuk memiliki aturan perhitungan yang independen sehingga perubahan pada satu jalur tidak akan memengaruhi jalur lainnya. Pendekatan ini membuat sistem lebih fleksibel, mudah dikembangkan, serta mendukung penerapan Strategy Pattern dan Open Closed Principle (OCP) karena penambahan jalur baru dapat dilakukan dengan membuat class baru tanpa mengubah kode yang sudah ada.

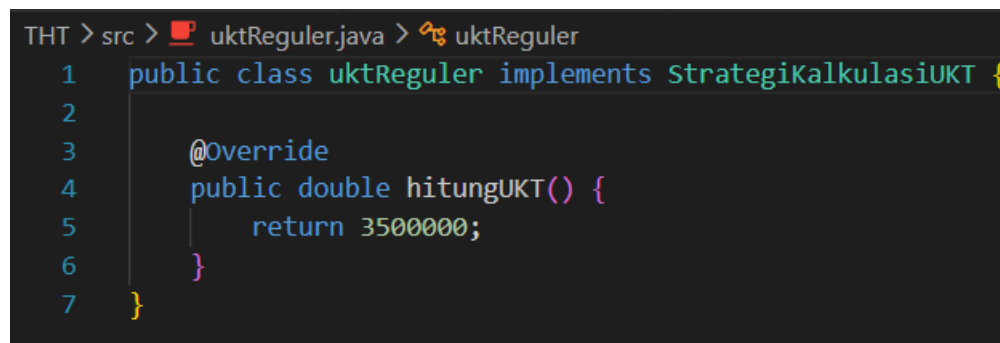
13.

```
THT > src > uktMbkm.java > uktMbkm
1  public class uktMbkm implements StrategiKalkulasiUKT {
2
3      @Override
4      public double hitungUKT() {
5          return 2500000;
6      }
7  }
```

Penjelasan:

Pada bagian ini saya membuat class `uktMbkm` yang mengimplementasikan interface `StrategiKalkulasiUKT`. Class ini digunakan untuk merepresentasikan skema perhitungan Uang Kuliah Tunggal (UKT) bagi mahasiswa yang mengikuti program Merdeka Belajar Kampus Merdeka (MBKM). Karena mengimplementasikan interface `StrategiKalkulasiUKT`, class ini wajib menyediakan implementasi untuk method `hitungUKT()`. Pada method tersebut, saya mengembalikan nilai 2500000 yang menunjukkan bahwa mahasiswa program MBKM dikenakan UKT sebesar Rp2.500.000. Nilai ini merupakan simulasi kebijakan UKT khusus yang berbeda dari jalur mahasiswa lainnya. Dengan memisahkan perhitungan UKT MBKM ke dalam class tersendiri, sistem dapat mendukung penambahan program atau jalur baru tanpa harus mengubah kode perhitungan UKT yang sudah ada. Pendekatan ini membuat sistem lebih mudah dikembangkan, mengurangi risiko terjadinya bug akibat perubahan kode, serta mendukung penerapan Strategy Pattern dan Open Closed Principle (OCP) karena sistem terbuka untuk penambahan fitur baru namun tertutup terhadap modifikasi kode yang sudah berjalan dengan baik.

14.



```
THT > src > uktReguler.java > uktReguler
1 public class uktReguler implements StrategiKalkulasiUKT {
2
3     @Override
4     public double hitungUKT() {
5         return 3500000;
6     }
7 }
```

Penjelasan:

Pada bagian ini saya membuat class `uktReguler` yang mengimplementasikan interface `StrategiKalkulasiUKT`. Class ini digunakan untuk merepresentasikan skema perhitungan Uang Kuliah Tunggal (UKT) bagi mahasiswa jalur Reguler. Karena mengimplementasikan interface `StrategiKalkulasiUKT`, class ini wajib menyediakan implementasi untuk method `hitungUKT()`. Pada implementasinya, method tersebut mengembalikan nilai 3500000 yang menunjukkan bahwa mahasiswa jalur reguler dikenakan UKT sebesar Rp3.500.000. Nilai ini merupakan simulasi biaya kuliah yang ditetapkan untuk mahasiswa reguler dan dapat disesuaikan dengan kebijakan kampus yang berlaku. Dengan memisahkan logika perhitungan UKT ke dalam class tersendiri, sistem menjadi lebih terstruktur dan setiap jalur mahasiswa memiliki aturan perhitungan yang independen. Pendekatan ini memudahkan proses pemeliharaan serta pengembangan sistem karena perubahan pada satu skema UKT tidak akan memengaruhi skema lainnya. Selain itu, penggunaan class ini mendukung penerapan Strategy Pattern dan Open Closed Principle (OCP) karena penambahan jalur mahasiswa

baru dapat dilakukan dengan membuat class baru tanpa harus mengubah kode yang sudah ada.

15.

```
THT > src > krsPdfPrinter.java > krsPdfPrinter
1 public class krsPdfPrinter {
2
3     public void cetakPDF() {
4         System.out.println(x: "PDF KRS berhasil dibuat.");
5     }
6 }
```

Penjelasan:

Pada bagian ini saya membuat class `krsPdfPrinter` yang bertanggung jawab untuk mencetak atau menghasilkan dokumen Kartu Rencana Studi (KRS) dalam format PDF. Class ini memiliki satu method yaitu `cetakPDF()` yang digunakan untuk menjalankan proses pembuatan dokumen PDF. Pada implementasi saat ini, method tersebut menampilkan pesan *"PDF KRS berhasil dibuat."* sebagai simulasi bahwa proses pembuatan dokumen telah berhasil dilakukan. Pemisahan proses pencetakan PDF ke dalam class tersendiri merupakan penerapan Single Responsibility Principle (SRP) karena class ini hanya memiliki satu tanggung jawab, yaitu mengelola pembuatan dokumen KRS. Dengan desain seperti ini, apabila di kemudian hari diperlukan perubahan format PDF, penambahan kop surat, tanda tangan digital, atau fitur pencetakan lainnya, pengembang hanya perlu melakukan perubahan pada class `krsPdfPrinter` tanpa memengaruhi proses validasi, perhitungan UKT, maupun penyimpanan data. Pendekatan ini membuat sistem lebih modular, mudah dipelihara, dan mengurangi risiko terjadinya kesalahan pada komponen lain ketika dilakukan pengembangan fitur.

16.

```
THT > src > krsRepository.java > krsRepository
1 public class krsRepository {
2
3     private DatabaseConnection database;
4
5     public krsRepository(DatabaseConnection database) {
6         this.database = database;
7     }
8
9     public void simpan(String data) {
10        database.saveKRS(data);
11    }
12 }
```

Penjelasan:

Pada bagian ini saya membuat class `krsRepository` yang berfungsi sebagai penghubung antara sistem dan database dalam proses penyimpanan data KRS. Class ini memiliki atribut database yang bertipe `DatabaseConnection`, yaitu sebuah interface yang digunakan sebagai abstraksi koneksi database. Pada constructor `krsRepository(DatabaseConnection database)`, saya melakukan Dependency Injection dengan menerima objek database dari luar class dan menyimpannya ke dalam atribut database. Selanjutnya, saya membuat method `simpan(String data)` yang bertugas menyimpan data KRS dengan memanggil method `saveKRS(data)` milik objek database yang digunakan. Dengan cara ini, class `krsRepository` tidak bergantung pada implementasi database tertentu seperti MySQL atau Cloud NoSQL, melainkan hanya bergantung pada interface `DatabaseConnection`. Pendekatan ini mendukung penerapan Dependency Inversion Principle (DIP) karena modul tingkat tinggi tidak bergantung pada modul tingkat rendah secara langsung. Selain itu, desain ini juga membuat sistem lebih fleksibel, sehingga proses migrasi database atau penambahan jenis database baru dapat dilakukan tanpa perlu mengubah kode pada class `krsRepository`.

17.

```
THT > src > krsService.java > krsService
1  public class krsService {
2
3      private krsValidator validator;
4      private krsRepository repository;
5      private krsPdfPrinter printer;
6      private StrategiKalkulasiUKT strategi;
7
8      public krsService(
9          krsValidator validator,
10         krsRepository repository,
11         krsPdfPrinter printer,
12         StrategiKalkulasiUKT strategi) {
13
14         this.validator = validator;
15         this.repository = repository;
16         this.printer = printer;
17         this.strategi = strategi;
18     }
19
20     public void prosesKRS(int totalSKS) {
21
22         if(validator.validasiSKS(totalSKS)) {
23
24             double ukt = strategi.hitungUKT();
25
26             System.out.println("UKT : Rp " + ukt);
27
28             repository.simpan(data: "Data KRS");
29
30             printer.cetakPDF();
31         }
32     }
33 }
```

Penjelasan:

Pada bagian ini saya membuat class `krsService` yang berfungsi sebagai pusat logika bisnis dalam proses pengelolaan Kartu Rencana Studi (KRS). Class ini bertugas mengoordinasikan berbagai komponen sistem, yaitu `krsValidator` untuk melakukan validasi data, `krsRepository` untuk menyimpan data ke database, `krsPdfPrinter` untuk mencetak dokumen PDF, serta `StrategiKalkulasiUKT` untuk menghitung besaran UKT mahasiswa. Keempat komponen tersebut disimpan dalam atribut class dan diinisialisasi melalui constructor `krsService()` menggunakan teknik Dependency Injection, sehingga class ini tidak membuat objeknya sendiri dan menjadi lebih fleksibel serta mudah diuji.

Selanjutnya, saya membuat method `prosesKRS(int totalSKS)` yang digunakan untuk menjalankan seluruh alur proses KRS. Method ini pertama-tama memanggil `validator.validasiSKS(totalSKS)` untuk memastikan jumlah SKS yang diambil mahasiswa sesuai dengan aturan yang berlaku. Jika validasi berhasil, sistem akan menghitung UKT menggunakan method `hitungUKT()` dari objek strategi yang dipilih, kemudian menampilkan hasil perhitungan UKT kepada pengguna. Setelah itu, data KRS disimpan ke database melalui method `repository.simpan()`, dan terakhir sistem memanggil `printer.cetakPDF()` untuk menghasilkan dokumen KRS dalam format PDF.

Dengan memisahkan setiap tanggung jawab ke dalam class yang berbeda, `krsService` hanya berfokus pada pengaturan alur proses bisnis tanpa menangani detail implementasi masing-masing komponen. Pendekatan ini mendukung penerapan Single Responsibility Principle (SRP) dan Dependency Inversion Principle (DIP) sehingga sistem menjadi lebih modular, mudah dipelihara, serta lebih siap menghadapi perubahan kebutuhan di masa mendatang.

18.

```
THT > src > krsValidator.java > krsValidator
1  public class krsValidator {
2
3      public boolean validasiSKS(int totalSKS) {
4
5          if(totalSKS > 24) {
6              System.out.println(x: "SKS melebihi batas.");
7              return false;
8          }
9
10         return true;
11     }
12 }
```

Penjelasan:

Pada bagian ini saya membuat class `krsValidator` yang berfungsi untuk melakukan validasi terhadap jumlah SKS yang diambil oleh mahasiswa saat proses pengisian Kartu Rencana Studi (KRS). Class ini memiliki satu method yaitu

validasiSKS(int totalSKS) yang menerima parameter berupa jumlah total SKS yang dipilih mahasiswa. Di dalam method tersebut, sistem melakukan pengecekan apakah jumlah SKS melebihi batas maksimum yang telah ditentukan, yaitu 24 SKS. Jika jumlah SKS lebih dari 24, sistem akan menampilkan pesan *"SKS melebihi batas."* dan mengembalikan nilai false yang menandakan bahwa data tidak valid. Sebaliknya, jika jumlah SKS masih berada dalam batas yang diperbolehkan, method akan mengembalikan nilai true sehingga proses KRS dapat dilanjutkan.

Pemisahan proses validasi ke dalam class krsValidator merupakan penerapan Single Responsibility Principle (SRP) karena class ini hanya memiliki satu tanggung jawab, yaitu memvalidasi aturan pengambilan SKS mahasiswa. Dengan desain seperti ini, apabila di masa mendatang terdapat perubahan kebijakan akademik, misalnya batas maksimum SKS berubah atau ditambahkan aturan validasi baru, pengembang cukup melakukan perubahan pada class krsValidator tanpa memengaruhi komponen lain seperti penyimpanan data, perhitungan UKT, maupun pencetakan PDF. Pendekatan ini membuat sistem lebih terstruktur, mudah dipelihara, dan lebih fleksibel terhadap perubahan kebutuhan.

19.

```
THT > src > Main.java > Main > main(String[])
1  public class Main {
2
3      Run | Debug
4      public static void main(String[] args) {
5          DatabaseConnection db =
6              new CloudNoSQLConnection();
7          krsValidator validator =
8              new krsValidator();
9          krsRepository repository =
10             new krsRepository(db);
11          krsPdfPrinter printer =
12             new krsPdfPrinter();
13          StrategiKalkulasiUKT strategi =
14             new uktMbkM();
15          krsService service =
16             new krsService(
17                 validator,
18                 repository,
19                 printer,
20                 strategi
21             );
22          MataKuliah mk1 =
23             new MataKuliahTeori(
24                 nama: "Pemrograman Berorientasi Objek",
25                 sks: 3);
26          MataKuliah mk2 =
27             new MataKuliahPraktikum(
28                 nama: "Praktikum PBO",
29                 sks: 1);
30          MataKuliah mk3 =
31             new MataKuliahKKN(
32                 nama: "Kuliah Kerja Nyata",
33                 sks: 4);
34          mk1.tampilkanInfo();
35          mk2.tampilkanInfo();
36          mk3.tampilkanInfo();
37          service.prosesKRS(totalSKS: 20);
38      }
39  }
```

Penjelasan:

Pada bagian ini saya membuat class Main yang berfungsi sebagai titik awal eksekusi program sekaligus sebagai tempat untuk menguji seluruh komponen yang telah dibuat pada sistem SIAKAD. Di dalam method main(), saya terlebih dahulu membuat objek CloudNoSQLConnection yang disimpan ke dalam variabel db sebagai implementasi koneksi database yang akan digunakan sistem. Selanjutnya, saya membuat objek krsValidator, krsRepository, krsPdfPrinter, dan uktMbkm yang masing-masing berfungsi untuk melakukan validasi KRS, menyimpan data, mencetak PDF, serta menghitung UKT berdasarkan jalur MBKM. Semua objek tersebut kemudian diberikan ke constructor krsService sehingga seluruh proses bisnis KRS dapat dijalankan melalui satu class layanan.

Setelah itu, saya membuat tiga objek mata kuliah yaitu MataKuliahTeori, MataKuliahPraktikum, dan MataKuliahKKN yang masing-masing merepresentasikan jenis mata kuliah yang berbeda. Ketiga objek tersebut disimpan menggunakan tipe data induk MataKuliah, yang menunjukkan penerapan konsep Polymorphism. Kemudian program memanggil method tampilkanInfo() pada masing-masing objek untuk menampilkan informasi mata kuliah sesuai implementasi yang dimiliki oleh class turunannya.

Pada bagian akhir, program memanggil method service.prosesKRS(20) untuk menjalankan seluruh alur proses KRS dengan total pengambilan sebanyak 20 SKS. Method ini akan melakukan validasi jumlah SKS, menghitung UKT mahasiswa menggunakan strategi MBKM, menyimpan data KRS ke database Cloud NoSQL, dan mencetak dokumen PDF KRS. Dengan adanya class Main, seluruh komponen yang telah dirancang dapat diintegrasikan dan dijalankan secara bersama-sama sehingga dapat menunjukkan penerapan prinsip-prinsip SOLID, Inheritance, Polymorphism, Abstraction, serta Dependency Injection dalam satu sistem yang terstruktur dan mudah dikembangkan.

V. Kesimpulan

Berdasarkan hasil analisis dan perancangan ulang sistem SIAKAD, saya berhasil mengidentifikasi berbagai permasalahan arsitektur yang berasal dari pelanggaran prinsip SOLID. Sistem lama memiliki kelas yang terlalu besar, ketergantungan langsung terhadap database tertentu, serta desain antarmuka yang tidak sesuai dengan kebutuhan masing-masing jenis mata kuliah.

Untuk mengatasi masalah tersebut, saya menerapkan lima prinsip SOLID yaitu Single Responsibility Principle (SRP), Open Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP). Hasil refactoring menghasilkan sistem yang lebih modular, fleksibel, mudah dipelihara, dan siap dikembangkan di masa depan.

Selain itu, sistem yang baru juga mampu mendukung migrasi database dari MySQL ke Cloud NoSQL serta penambahan jalur mahasiswa baru seperti MBKM tanpa perlu mengubah kode yang sudah berjalan. Dengan demikian kualitas perangkat lunak meningkat dan risiko terjadinya kerusakan akibat perubahan dapat diminimalkan.

VI. Daftar Pustaka

Oracle. (2023). *Java Documentation*. <https://docs.oracle.com/en/java/>